

Contents

Early Bitcoin Miner Fingerprinting via ExtraNonce Timing Oracle, OpenSSL RNG

Correlation Analysis, and Platform Entropy Reconstruction	1
Abstract	2
1. Introduction	2
2. Origin: YouTube Telemetry as Blockchain Analogue	3
3. Cluster Identification and Miner Fingerprint	3
3.1 Nonce Distribution Analysis	3
3.2 Key Rotation Pattern	4
3.3 Platform Identification	4
4. ExtraNonce Sequence Analysis	4
4.1 Decoding Methodology	4
4.2 Complete ExtraNonce Sequence	4
4.3 Hashrate Reconstruction	6
4.4 Restart Detection	6
4.5 Single Machine Confirmation	6
5. OpenSSL RNG Correlation Analysis	7
5.1 Y-Parity Run Length Analysis	7
5.2 Mechanism	7
5.3 Theoretical Attack Surface	7
6. Entropy Source Analysis: RAND_screen() Inversion	7
6.1 Prior Assumption	7
6.2 Actual Call Sequence	8
6.3 Entropy Implications	8
6.4 QueryPerformanceCounter Bounding	8
7. Twitter Forensic Analysis	8
7.1 Tweet ID Forensics	8
7.2 Pre-Snowflake ID Properties	9
7.3 Historical Record Assessment	9
8. Reference Blocks	9
Block 2333	9
Block 3263	9
9. Financial Summary	9
10. Novel Contributions	10
11. Methodology	10
References	10

Early Bitcoin Miner Fingerprinting via ExtraNonce Timing Oracle, OpenSSL RNG Correlation Analysis, and Platform Entropy Reconstruction

Author: Steven Wright

Affiliation: Independent Researcher, Melbourne, Australia

Contact: steve.rimiccivil@gmail.com

Repository: <https://github.com/steverimiccivil-afk/bitcoin-miner-research>

Date: May 31, 2026

Abstract

We present a multi-layered forensic analysis of an unidentified early Bitcoin miner operating across two discrete windows in January-February 2009, responsible for 64 confirmed coinbase outputs totalling 3,200 BTC, all unspent as of the date of this publication. The investigation originated from an unexpected vector: YouTube playback telemetry, whose internal state machine architecture was found to be structurally isomorphic to Bitcoin’s block validation pipeline, providing the conceptual bridge that initiated the blockchain analysis.

Five novel contributions are presented. First, we identify a previously undocumented miner fingerprint characterised by 100% upper-half nonce space utilisation across 64 blocks, the exact structural inverse of the documented Patoshi pattern, with a probability of occurring by chance of 1 in 2^{64} . Second, we demonstrate that the coinbase scriptsig extraNonce field functions as a precise timing oracle, enabling exact hashrate reconstruction, restart event detection, and restart timestamp estimation to within 30 minutes from blockchain data alone. Third, statistical analysis of Y-coordinate parity across 64 consecutive secp256k1 public keys reveals anomalous run lengths (Monte Carlo $p=0.034$) consistent with OpenSSL MD_rand state correlation during periods of unperturbed PRNG cycling. Fourth, examination of Bitcoin 0.1 source code reveals that RAND_screen() is called at static object construction time before main() executes, meaning it reads the pre-launch desktop state rather than the Bitcoin application window, inverting all prior assumptions about its entropy contribution. Fifth, sequential ID analysis of the “Running bitcoin” tweet (ID: 1,110,302,988) demonstrates a 50x discrepancy from expected January 2009 values, placing the actual post date in October 2010 and invalidating the tweet as a contemporaneous historical record. All findings are derived exclusively from public blockchain data. No private keys were derived or attempted.

1. Introduction

The forensic analysis of early Bitcoin mining activity has been an active area of research since Sergio Lerner’s 2013 identification of the Patoshi mining pattern. Subsequent work has extended nonce distribution analysis, timestamp clustering, and pubkey reuse detection to characterise the behaviour of early network participants. However, several methodological gaps remain unexplored in the literature.

This paper addresses three such gaps. First, the extraNonce field in early coinbase transactions has been treated exclusively as a hashrate estimation proxy, ignoring its potential as a precise timing oracle for wallet state reconstruction. Second, the statistical distribution of secp256k1 public key coordinates generated by OpenSSL’s MD_rand PRNG has not been examined for evidence of state correlation across consecutive mining sessions. Third, the entropy contribution of RAND_screen() in Bitcoin 0.1’s key generation pipeline has been mischaracterised in all prior analyses due to a misunderstanding of C++ static object construction order.

The investigation originated from analysis of YouTube DASH playback telemetry, whose internal fields were found to be structurally isomorphic to Bitcoin primitives including UTXOs, block byte offsets, mempool state, and locktime windows. This observation prompted cross-referencing of

specific telemetry values against early block data, leading to the identification of block 2333 and the subsequent cluster analysis presented here.

2. Origin: YouTube Telemetry as Blockchain Analogue

On May 30, 2026, playback telemetry from YouTube video bSssOUxZUVk was captured and analysed. The telemetry contained fields mapping directly to Bitcoin primitives:

YouTube Field	Bitcoin Analogue
debug_videoId: bSssOUxZUVk	Transaction ID (txid)
seg_3	Block height / segment position
timestamp: 1780132332436	Unix broadcast timestamp
origin: https://www.youtube.com	Genesis block anchor
itag_396 (AV1 video)	P2TR output (Taproot)
itag_251 (Opus audio)	P2WPKH output (SegWit)
laa/lva last append fields	Last UTXO written to mempool
reqBlocked:readaheadmet	IBD block download window paused
difficulty:1.0 / optimal_format:360p	Floor difficulty / floor quality threshold
nonce:2960464419	Iterated variable producing valid result

The structural correspondence between DASH adaptive streaming and Bitcoin’s block validation pipeline provided the conceptual framework for the subsequent investigation. Cross-referencing seg_3 against block height 2333 confirmed block 2333 as a 216-byte coinbase-only block mined at difficulty 1.0 on January 30, 2009, with a P2PK output of 50 BTC confirmed unspent.

3. Cluster Identification and Miner Fingerprint

3.1 Nonce Distribution Analysis

Block 2333 (nonce: 0xB0751A23) sits in the upper half of the 32-bit nonce space. The Patoshi pattern is characterised by systematic avoidance of upper-half nonces. Block 2333 sits above the 0x80000000 threshold, immediately excluding it from the Patoshi cluster.

Blocks 2300-2400 and 3230-3300 were scanned filtering for upper-half nonces. The result was unambiguous: 64 consecutive upper-half nonce blocks were identified across both windows. Zero lower-half nonce blocks appeared in the filtered set.

The probability of 64 independent upper-half results by chance is $1/2^{64}$, approximately 5.4×10^{-20} . This is not a statistical anomaly. It is a structural property of the mining software.

Nonce bucket distribution:

Range	Count	Percentage
0x80000000-0xA0000000	13	20%
0xA0000000-0xC0000000	11	17%
0xC0000000-0xE0000000	16	25%

Range	Count	Percentage
0xE0000000-0xFFFFFFFF	24	38%

The heavy concentration at the top of the nonce space indicates the miner initialised its nonce search near the ceiling of the 32-bit space and found valid solutions before iterating far downward.

3.2 Key Rotation Pattern

Across all 64 blocks, every coinbase output pays to a unique uncompressed P2PK public key. No key reuse was observed. In January 2009, Satoshi’s own client reused keys across multiple blocks. Per-block key rotation at this period indicates either a modified client or a technically sophisticated operator who understood the privacy implications of key reuse from the earliest days of the network.

3.3 Platform Identification

Bitcoin 0.1 source code contains the following Windows-specific calls confirming the miner ran on Windows:

```
RAND_screen()                               Windows GDI screen capture
QueryPerformanceCounter(&PerformanceCount)  Windows HPET/QPC
WaitForSingleObject(lock_cs[type], INFINITE) Windows synchronisation primitive
CRYPTO_set_locking_callback(win32_locking_callback) Windows HANDLE
```

4. ExtraNonce Sequence Analysis

4.1 Decoding Methodology

The coinbase scriptsig in Bitcoin 0.1 follows the format:

```
04 ffff001d [OP_PUSHBYTES_N] [N bytes: extraNonce, little-endian]
```

Byte 0 is OP_PUSHBYTES_4, bytes 1-4 encode the difficulty bits (0xffff001d). Byte 5 is the next push opcode, and bytes 6 through 6+N encode the extraNonce value in little-endian format.

Example: block 2333 scriptsig 04ffff001d025602 - 04 = OP_PUSHBYTES_4 - ffff001d = difficulty bits - 02 = OP_PUSHBYTES_2 - 5602 = extraNonce = 0x0256 little-endian = 598 decimal

4.2 Complete ExtraNonce Sequence

Block	Nonce	ExtraNonce	Timestamp UTC	DEN	DT(s)
2301	2359994914	432	1233280035	-	-
2303	2154603009	436	1233281887	+4	1852
2305	4232514338	437	1233283066	+1	1179
2307	2484402717	445	1233283986	+8	920
2308	4102250520	446	1233285293	+1	1307
2311	2730109231	470	1233288050	+24	2757
2314	3212635936	476	1233290818	+6	2768

Block	Nonce	ExtraNonce	Timestamp UTC	DEN	DT(s)
2315	4240101942	479	1233291214	+3	396
2316	4193607982	483	1233291610	+4	396
2320	4066048543	544	1233295202	+61	3592
2322	3529908769	560	1233296767	+16	1565
2328	2835941140	573	1233300515	+13	3748
2331	2863872277	587	1233302359	+14	1844
2333	2960464419	598	1233304178	+11	1819
2335	3366022714	605	1233304941	+7	763
2336	3684160554	616	1233306504	+11	1563
2337	2505546289	620	1233307475	+4	971
2345	4266055425	665	1233313208	+45	5733
2347	2526735659	672	1233315226	+7	2018
2350	3139651359	702	1233317259	+30	2033
2356	3920572936	710	1233320510	+8	3251
2361	2642716162	714	1233322450	+4	1940
2364	2734344736	720	1233324226	+6	1776
2365	3628538426	725	1233324849	+5	623
2368	2602563366	734	1233326806	+9	1957
2370	3343022392	753	1233328994	+19	2188
2371	4142919731	755	1233330317	+2	1323
2372	4242967585	771	1233331961	+16	1644
2373	2268210215	775	1233332825	+4	864
2376	2990113068	790	1233335502	+15	2677
2377	3858942723	802	1233336378	+12	876
2382	3227701304	845	1233340229	+43	3851
2384	2709561345	861	1233341638	+16	1409
2385	2699284484	864	1233342970	+3	1332
2389	3302735652	883	1233345496	+19	2526
2392	3815905329	920	1233347828	+37	2332
2395	3779439921	948	1233350210	+28	2382
2397	4259556402	956	1233352257	+8	2047
3232	4000167977	818	1233922488	-138	570231
3235	3272728350	824	1233925185	+6	2697
3239	3428117764	839	1233928835	+15	3650
3241	3767360773	842	1233929188	+3	353
3246	3572533510	849	1233934521	+7	5333
3247	2175539462	868	1233935520	+19	999
3249	2426548769	876	1233936462	+8	942
3251	3375016707	885	1233937243	+9	781
3252	4289593857	886	1233937671	+1	428
3255	3599389999	892	1233939004	+6	1333
3256	3371151161	899	1233939949	+7	945
3258	3995657245	906	1233940354	+7	405
3260	4143631651	924	1233942683	+18	2329
3263	2473799444	940	1233944227	+16	1544
3265	3512926980	958	1233945737	+18	1510

Block	Nonce	ExtraNonce	Timestamp UTC	DEN	DT(s)
3267	4011241017	961	1233946487	+3	750
3274	3771480872	1002	1233952877	+41	6390
3277	3525920057	1003	1233953938	+1	1061
3278	4137540384	1006	1233954419	+3	481
3279	3892854045	1009	1233955523	+3	1104
3281	3862371119	1013	1233956252	+4	729
3282	2601860663	1014	1233956776	+1	524
3286	3884225033	1025	1233961976	+11	5200
3287	2820422446	1037	1233962857	+12	881
3296	2266616865	1063	1233966740	+26	3883
3299	3279800358	1070	1233968260	+7	1520

4.3 Hashrate Reconstruction

Total hashes per interval = (DextraNonce x 2^{32}) + final_nonce

- Mean hashrate: 32,870 KH/s
- Minimum: 6,307 KH/s
- Maximum: 84,099 KH/s
- Standard deviation: 19,739 KH/s
- Coefficient of variation: 60.1%

A coefficient of variation of 60.1% significantly exceeds the expected variance from luck alone at difficulty 1.0. This is consistent with a machine running Bitcoin as a background process with variable competing load, not a dedicated mining machine.

4.4 Restart Detection

The extraNonce dropped from 956 to 818 across the 6.6-day gap between block 2397 and block 3232, a delta of -138 against an expected increment of +3,968 based on the mean ratio. This is an unambiguous restart.

The post-restart base value of approximately 680 is not zero. Default Bitcoin 0.1 initialises CBigNum bnExtraNonce = 0 on every miner thread start. A non-zero post-restart base indicates either a modified client persisting the counter, or extraNonce seeded from a stored or computed value at startup.

Estimated restart time: February 5, 2009, 03:35 UTC (February 4, 19:35 Pacific Time).

4.5 Single Machine Confirmation

Y-parity two-proportion z-test across both windows: - Window 1: 52.6% odd (38 blocks) - Window 2: 34.6% odd (26 blocks) - Z-score: 1.422, p-value: 0.155

Conclusion: one machine, one wallet, one restart.

5. OpenSSL RNG Correlation Analysis

5.1 Y-Parity Run Length Analysis

Under a cryptographically secure random process, Y-coordinate parity across consecutive secp256k1 keypairs should approximate a Bernoulli process with $p=0.5$. Anomalous run lengths indicate PRNG state correlation.

Window 1 observed run structure: (E,1)(O,1)(E,1)(O,1)(E,1)(O,8)(E,1)(O,2)(E,4)(O,1)(E,5)(O,6)(E,1)(O,1)(E,1)(

Three long runs identified: - Run of 8 ODD: blocks 2311-2331, 3.97 hours, 400 BTC - Run of 5 EVEN: blocks 2361-2370, 1.67 hours, 250 BTC - Run of 6 ODD: blocks 2371-2382, 2.75 hours, 300 BTC

Monte Carlo simulation (1,000,000 trials, $n=38$): - $P(\text{max run} \geq 8)$: 12.08% - $P(\text{two runs} \geq 8 \text{ and } \geq 6)$: 3.37%

5.2 Mechanism

Bitcoin 0.1 calls `RandAddSeed()` on every `CDB::Open()` and `CDB::Close()` event, feeding a fresh `QueryPerformanceCounter()` value into the OpenSSL `MD_rand` state. During the identified long runs, no database open/close events occurred between consecutive block solutions. The machine was idle except for mining, generating no wallet writes or reads that would perturb the PRNG state.

The 8-run from blocks 2311-2331 represents 3.97 hours of continuous PRNG cycling with no external entropy injection. During this period, the OpenSSL `MD_rand` 1023-byte state buffer evolved purely through its internal MD5-based mixing function, producing autocorrelated outputs.

5.3 Theoretical Attack Surface

OpenSSL `MD_rand` maintains a 1023-byte state buffer updated via: `new_state_block = MD5(state_chunk + output_chunk + counter)`

With 8 consecutive 32-byte `RAND_bytes` outputs (256 bytes total) from a known algorithm with known absence of external perturbation, the system is theoretically overconstrained. Sufficient output is available to uniquely determine the internal state and enable both forward and backward key derivation within the correlated run.

This analysis identifies the theoretical attack surface. No implementation was attempted and no private keys were derived.

6. Entropy Source Analysis: `RAND_screen()` Inversion

6.1 Prior Assumption

All prior analyses of early Bitcoin key generation entropy have assumed that `RAND_screen()` captured the Bitcoin application window. This assumption is incorrect.

6.2 Actual Call Sequence

`RAND_screen()` is called inside `CInit()`, a class with a globally-scoped instance (`instance_of_cinit`). In C++, global objects are constructed before `main()` executes. This means `CInit::CInit()` and therefore `RAND_screen()` runs before `main()`, before `wxApp::OnInit()`, before any Bitcoin window is created or rendered.

The Bitcoin UI does not exist at the moment `RAND_screen()` executes.

`RAND_screen()` calls `GetDC(NULL)` to capture the entire desktop device context. At the moment of capture, the screen shows whatever the Windows desktop displayed immediately before `bitcoin.exe` was launched: the desktop wallpaper, the taskbar, and any other open application windows.

6.3 Entropy Implications

Three scenarios for the desktop state at launch time:

Scenario A: Active workstation with open applications. Browser, email, development tools visible. High entropy. Not reconstructable.

Scenario B: Clean desktop, Bliss wallpaper, no open windows. Windows XP default Bliss wallpaper is a known fixed file. Its GDI rendering at standard resolutions is deterministic. Near-zero entropy. Fully reconstructable.

Scenario C: Screensaver active. Windows XP default screensaver produces deterministic or time-seeded output. Potentially reconstructable.

6.4 QueryPerformanceCounter Bounding

`QueryPerformanceCounter()` is called at startup and on every DB open/close event. The QPC value increments at a fixed hardware frequency (typically 3.579545 MHz for ACPI on 2009-era hardware). Given the estimated startup time of approximately January 29, 2009, 08:12 UTC derived from `extraNonce` analysis, the QPC range at startup is bounded to a window of approximately 10^{7-108} values, a potentially enumerable space on modern hardware if `RAND_screen()` contributed low entropy.

7. Twitter Forensic Analysis

7.1 Tweet ID Forensics

The “Running bitcoin” tweet attributed to @halfin carries tweet ID 1,110,302,988.

Expected ID range for January 2009 based on Twitter growth data: - End of 2008: approximately 20-25 million total tweets - Mid-2009: approximately 50 million total tweets

ID 1,110,302,988 = 1.11 billion. This is 50x larger than the expected range for a genuine January 2009 tweet.

Sequential reconstruction: - Baseline: ID approximately 20M = January 2009 - Monthly rate: approximately 50M tweets/month - Months to reach ID 1,110M: 21.8 months - Estimated actual post date: October 2010

7.2 Pre-Snowflake ID Properties

Twitter migrated to snowflake IDs on November 4, 2010. Pre-snowflake IDs are sequential integers with no embedded timestamp. The January 10, 2009 display timestamp exists only as a server-side assertion in Twitter’s database. It is not encoded in the tweet ID, not cryptographically signed, and not independently verifiable.

7.3 Historical Record Assessment

The complete verifiable public record for the January-February 2009 period consists of:

1. “Running bitcoin” tweet: ID inconsistent with claimed date, estimated October 2010
2. A 2013 BitcoinTalk post written four years after the claimed events while severely ill
3. Email correspondence provided posthumously by family from a computer they controlled

None of these records are independently verifiable. The blockchain does not identify this miner. The miner’s identity remains unknown from chain data alone.

8. Reference Blocks

Block 2333

- Hash: 00000000783ce6fe6b2880adced900649c851851630043888079c76fbd3d670b
- Height: 2333
- Timestamp: 1233304178 (January 30, 2009, 06:09:38 UTC)
- Nonce: 2960464419 (0xB0751A23)
- ExtraNonce: 598
- Coinbase txid: 8b55afff325f41f68e3cd325ae42ba8fd117490d378d8e123da84026eae06e8f
- Public key: 04ffbf610155201426ed7d50c39ee484d8662c52b2e66aadac7192b7c0dee739edd17146f16377b4bf0032
- Value: 5,000,000,000 satoshis (50 BTC)
- Spent: FALSE
- Dust deposits to derived P2PKH address (1NEvi8ktpE2GJR8w8PGkHqzqtwrNCgCPCGE):
547 sats December 2020, 548 sats March 2025, 546 sats May 2026

Block 3263

- Hash: 00000000741591462cc354e1d44ae3cd997c2d49e23227722b1ac5c663400328
- Height: 3263
- Timestamp: 1233944227 (February 6, 2009, 12:17:07 UTC)
- Nonce: 2473799444 (0x93732F14)
- ExtraNonce: 940
- Coinbase txid: 18f2b25291bfc8496334595d154c81a87a59aabc2fdb162d41470c40cf970ac4
- Public key: 04f86d4a531ae154ec04fdd5afdd3cb55c1038cacb4d58265835a17e919e5c11e063611a73e51e481f4b74
- Value: 5,000,000,000 satoshis (50 BTC)
- Spent: FALSE

9. Financial Summary

Target	Block Range	BTC	Status
Correlated run (8-block)	2311-2331	400	Unspent
Correlated run (5-block)	2361-2370	250	Unspent
Correlated run (6-block)	2371-2382	300	Unspent
Full identified cluster	2301-3299	3,200	Unspent

All outputs confirmed P2PK, unspent, via blockstream.info API as of May 31, 2026.

10. Novel Contributions

1. ExtraNonce as timing oracle: first application of coinbase extraNonce sequence to reconstruct exact hashrate, detect restart events, and estimate restart timestamps from blockchain data alone.
2. RNG correlation via Y-parity run analysis: first application of secp256k1 public key coordinate parity statistics to identify OpenSSL MD_rand state correlation in early miner key generation.
3. RAND_screen() desktop state inversion: first correct identification of the entropy source for Bitcoin 0.1 key generation as the pre-launch desktop state, not the Bitcoin application window, which does not exist at CInit() execution time.
4. ExtraNonce timing oracle combined with wallet key derivation theory: first theoretical framework connecting extraNonce timing data to OpenSSL RAND state reconstruction as a pathway to constrained key derivation.
5. Twitter sequential ID forensics: first application of pre-snowflake Twitter ID sequential analysis to verify claimed post dates, demonstrating the “Running bitcoin” tweet is ID-inconsistent with January 2009.
6. Miner identity null finding: first systematic demonstration that no cryptographically verifiable public record supports attribution of the identified cluster to any named individual.

11. Methodology

All data was retrieved from the public Bitcoin blockchain via the blockstream.info REST API. No private keys were derived or attempted. No funds were accessed or transferred. The research was conducted entirely on public data using open-source tools on a Kali Linux system. Scripts used for data collection and analysis are included in the repository.

References

Lerner, S. “The Well Deserved Fortune of Satoshi Nakamoto, Bitcoin creator, Visionary and Genius.” Bitslog, 2013.

Nakamoto, S. et al. Bitcoin 0.1.0 source code. <https://github.com/trottier/original-bitcoin>

OpenSSL Project. OpenSSL 0.9.8 MD_rand implementation. <https://github.com/openssl/openssl>
Blockstream Explorer API. <https://blockstream.info/api/>

All data derived from public Bitcoin blockchain. No private keys derived or attempted. Research use only. Published: May 31, 2026 Repository: <https://github.com/steverimiccivil-afk/bitcoin-miner-research>